

Contents

CMake	1
Структура CMake-файла	1
Этапы компиляции	1
1. Препроцессинг	2
2. Компиляция	3
3. Линковка	3
Организация растущего проекта: подкаталоги и таргеты	3
Содержимое bin/CMakeLists.txt	4
Содержимое lib/CMakeLists.txt	5
Дополнительные материалы	5
Файловый ввод-вывод	5

CMake

Структура CMake-файла

Проект на CMake обычно организован не одним файлом, а иерархией директорий, каждая из которых отвечает за свою часть проекта. Из конспекта:

```
.
├── CMakeLists.txt
├── bin
│   └── CMakeLists.txt
├── lib
│   └── CMakeLists.txt
└── tests
    └── CMakeLists.txt
```

Что это значит. CMakeLists.txt — это файл конфигурации сборки, который читает утилита cmake. В нём описывается, из каких файлов собирать программу, какие библиотеки к ней подключать, куда класть include-директории и т.д. Идея в том, что у проекта есть:

- **корневой** CMakeLists.txt — точка входа, из которой запускается сборка всего проекта;
- отдельные CMakeLists.txt в поддиректориях bin, lib, tests — каждый описывает сборку только своей части (исполняемого файла, библиотеки, тестов соответственно).

Такое разбиение называется по каталогам-**таргетам** (targets) — к этому понятию мы вернёмся ниже, когда будем разбирать пример с bin и lib.

Этапы компиляции

Компиляция программы на C/C++ — это не единый монолитный процесс, а последовательность из как минимум трёх различных этапов: **препроцессинг** → **компиляция** → **линковка**. Понимание этой цепочки важно, потому что многие ошибки сборки (например, “undefined reference” или “redefinition”) на самом деле относятся к конкретному этапу, и зная это, проще понять, что чинить.

1. Препроцессинг

Определение. Препроцессинг — это этап, на котором обрабатываются (подставляются, разворачиваются) **директивы препроцессора**: строки, начинающиеся с символа `#`. Препроцессор работает чисто текстово, ещё *до* того, как компилятор начинает разбирать язык C++ как таковой.

Примеры директив препроцессора: `#include` — вставляет содержимое указанного файла на место директивы; `#pragma` — служебная директива-инструкция компилятору (например, `#pragma once` для защиты от повторного включения заголовка); макросы — объявляются через `#define` и подставляются текстово везде, где встречается их имя.

Важное разделение: объявление vs определение.

Это центральная идея, вокруг которой строится вся организация файлов в C++:

- **Объявление (declaration)** — это то, что говорит компилятору: “такая-то сущность (функция, переменная, класс) существует, вот её сигнатура/тип”. Само по себе объявление не описывает, *что* делает функция.
- **Определение (definition)** — это то, что даёт “тело” сущности: конкретную реализацию функции, инициализацию переменной и т.д.

Отсюда — принятое соглашение в конспекте: **в .h-файлы пишем объявления, а в .cpp-файлы — определения.**

Разберём пример пошагово.

Объявление функции — идёт в заголовочный файл `.h`:

```
void myfunc();
```

Здесь мы просто говорим: “где-то существует функция `myfunc`, она ничего не принимает и ничего не возвращает (`void`)”. Тела у неё нет — это просто “контракт”.

Определение той же функции — идёт в `.cpp`-файл:

```
void myfunc() {  
}
```

Отличие ровно в одном: появилось тело функции (пусть даже пустое `{ }`). Это уже не просто обещание, что функция существует, а её реальная реализация.

Зачем вообще разделять на два файла? Затем, чтобы функцию можно было *использовать* в другом файле, не имея доступа к её реализации — достаточно знать только объявление (интерфейс). Поэтому создаётся отдельный файл `func.h`, в котором делается объявление функции, а в `main.cpp` (или любом другом файле, где функция нужна) это объявление просто подключается:

```
#include "func.h"
```

После этой строчки препроцессор *текстово* вставит содержимое `func.h` прямо в место, где написан `#include`. Компилятор, разбирая получившийся файл, увидит объявление `myfunc` и будет знать, что такая функция существует и как её вызывать (сигнатуру), даже если сама реализация лежит в другом `.cpp`-файле. Про то, где реализация физически находится, компилятор на этом этапе не заботится — за связывание вызова с реальным телом функции отвечает более поздний этап, линковка (см. ниже).

Важно не запутаться: объявление можно подключать (через `#include`) сколько угодно раз в разные файлы — это просто текст с “контрактом”. А вот определение той же функции в проекте должно быть только одно — иначе на этапе линковки компоновщик увидит два разных тела для одного и того же имени и выдаст ошибку.

2. Компиляция

Определение. На этапе компиляции каждый отдельный .cpp-файл (уже после того, как в нём развёрнуты все директивы препроцессора) превращается в **объектный файл** — `main.cpp` → `main.o`.

Что важно понимать про объектный файл:

- Он уже содержит машинный (или почти машинный) код для того, что определено в *этом конкретном* .cpp-файле.
- Но если в этом файле есть *вызовы* функций, которые определены в других .cpp-файлах (мы их только объявили через `#include "func.h"`, а не определили), то компилятор не может подставить туда реальный адрес — ведь он ещё не знает, где физически будет лежать код этой функции в итоговой программе.
- Вместо реального адреса компилятор оставляет **заглушку** — своего рода “метку-обещание”: здесь должен быть вызов функции с таким-то именем, а окончательный адрес допишется позже.

Про заглушку отдельно, поскольку в конспекте это подчёркнуто: заглушка — это название функции без адреса. Оно не обязано быть человекочитаемым в буквальном смысле (то есть это необязательно равно `myfunc`), но должно быть понятно **линкеру** — то есть однозначно идентифицировать функцию среди всех остальных функций в проекте. В конспекте приведён пример такого имени: `__MyFunc`. Такие “искажённые” (mangled) имена нужны, в частности, потому что в C++ функции могут быть перегружены (несколько функций с одинаковым именем, но разными параметрами), и одного лишь имени `myfunc` недостаточно, чтобы линкер понял, какую именно версию функции связывать — поэтому в реальное имя-заглушку кодируется дополнительная информация (например, о типах параметров).

Итог этапа: на выходе имеем набор .o-файлов, каждый из которых — это скомпилированный, но ещё не до конца “собранный вместе” кусок программы, где часть вызовов функций из других файлов представлена заглушками.

3. Линковка

Определение. На этапе линковки (компоновки) из уже готовых объектных файлов собирается один итоговый бинарь (исполняемый файл или библиотека).

Логическая связь с предыдущим этапом: именно на этом шаге все заглушки, оставленные компилятором (“здесь вызывается функция с таким-то именем”), заменяются на реальные адреса — потому что линкер видит все объектные файлы проекта разом и может сопоставить каждую заглушку с фактическим определением функции, которое нашлось в одном из .o-файлов (или в подключаемой библиотеке).

Частая ошибка, вытекающая отсюда: если вы объявили функцию (.h), подключили заголовок и вызвали функцию, но нигде не написали её определение (.cpp с телом) — компиляция каждого отдельного .cpp-файла пройдёт успешно (компилятору достаточно объявления, чтобы понять, как вызывать функцию, и он честно оставит заглушку). А вот линковка упадёт с ошибкой вида “undefined reference”: линкер обошёл все объектные файлы, но не нашёл ни одного определения для этой заглушки.

Организация растущего проекта: подкаталоги и таргеты

Когда проект небольшой, весь его `CMakeLists.txt` можно уместить в одном файле. Но по мере роста проекта такой файл становится большим и трудно читаемым. Поэтому

логические части проекта — в примере это `bin` (исполняемый файл), `lib` (библиотека) и `tests` (тесты) — выносят в отдельные подкаталоги, и каждая такая часть называется **таргетом** (target). У каждого таргета — свой собственный `CMakeLists.txt`, отвечающий только за сборку этой части.

А в корневом `CMakeLists.txt` эти подкаталоги просто подключаются, например:

```
add_subdirectory(bin)
```

Что делает эта команда (в конспекте помечено как непонятое место — разберём подробнее): `add_subdirectory(bin)` говорит CMake — “зайди в подкаталог `bin`, найди там его `CMakeLists.txt` и выполни (обработай) его как часть текущей сборки”. По сути это способ “включить” описание сборки одного таргета в общий процесс конфигурации проекта, не переписывая всё содержимое `bin/CMakeLists.txt` в корневой файл руками. Так же обычно подключаются и `lib`, и `tests` — по одной такой строчке на каждый подкаталог.

Содержимое `bin/CMakeLists.txt`

```
add_executable(...)
target_include_directories(hello
    PUBLIC
    ../lib)
target_link_library(hello
    PUBLIC
    lib)
```

Разберём построчно:

- **`add_executable(...)`** — объявляет новый таргет типа “исполняемый файл”: из перечисленных `.cpp`-файлов (в конспекте аргументы не выписаны полностью, отмечены многоточием) должен собраться бинарь, который можно запустить.
- **`target_include_directories(hello PUBLIC ../lib)`** — говорит: у таргета `hello` (то есть у нашего исполняемого файла) должна быть добавлена директория `../lib` в список путей поиска заголовочных файлов. Иными словами, когда в коде `hello` встретится `#include "func.h"`, компилятор будет знать, что этот файл нужно поискать в том числе в каталоге `../lib`. Слово `PUBLIC` означает, что эта настройка видимости заголовков распространяется не только на сам таргет `hello`, но и на всех, кто будет зависеть от `hello` дальше (в отличие от `PRIVATE`, где настройка осталась бы только внутри самого таргета).
- **`target_link_library(hello PUBLIC lib)`** — говорит компоновщику: чтобы собрать `hello`, нужно **слинковать** его с таргетом `lib` (то есть с библиотекой, описанной в соседнем подкаталоге `lib`). Это прямое отражение этапа линковки, разобранного выше: `hello` использует функции, объявленные в `../lib`, но определённые (реализованные) в `lib`, и без явного указания линковки линкер просто не будет знать, что нужно “заглянуть” в объектные файлы `lib`, чтобы найти определения для оставленных заглушек.

Дополнительное пояснение из конспекта: `func.a` — это условный пример **архива**: файла, который содержит внутри себя набор уже скомпилированных объектных файлов библиотеки. То есть `func.a` — это как бы “упаковка” из `func.o` (и возможно других `.o`-файлов), которую линкеру удобно подключать целиком как одну библиотеку, а не перечислять по отдельности каждый `.o`-файл.

Содержимое lib/CMakeLists.txt

```
add_library(lib func.cpp)
```

Здесь: - **lib** — это имя таргета (таргет типа “библиотека”); - **func.cpp** — это исходный файл, который нужно скомпилировать и включить в состав этого таргета.

То есть команда `add_library(lib func.cpp)` целиком аналогична по смыслу команде `add_executable`, только результатом сборки будет не исполняемый файл, а библиотека — тот самый архив вида `func.a`, который потом подключается через `target_link_library` в таргете `hello`.

Логическая связка всего примера: `bin/CMakeLists.txt` описывает исполняемый таргет `hello`, который зависит от библиотеки `lib` — как по заголовкам (`target_include_directories`, чтобы найти объявления), так и по скомпилированному коду (`target_link_library`, чтобы найти определения). А `lib/CMakeLists.txt` описывает саму библиотеку `lib`, собираемую из `func.cpp`. Именно это разделение на “объявление — библиотека предоставляет интерфейс через заголовки” и “определение — библиотека предоставляет реализацию через объектные файлы/архив” и есть практическое применение всей теории про препроцессинг/компиляцию/линковку, разобранной выше.

Дополнительные материалы

- Лекции по CMake: <https://is-itmo-c-26.github.io/lectures/videos>
 - Набор best practices для C++: isocpp.org (в конспекте отдельно отмечено, что там есть небольшие расхождения с Google C++ Style Guide — то есть эти два источника рекомендаций не всегда совпадают в деталях, и это нормально; следует ориентироваться на то, какой стандарт принят в конкретном проекте/команде)
-

Файловый ввод-вывод

Для работы с файлами в C++ используется заголовок:

```
#include <fstream>
```

Он предоставляет, среди прочего, два ключевых класса:

- `std::ifstream` (“input file stream”) — поток для **чтения** из файла;
- `std::ofstream` (“output file stream”) — поток для **записи** в файл.

Название хорошо отражает суть: `i` — input (вход, то есть чтение данных из файла в программу), `o` — output (выход, то есть запись данных из программы в файл).

Пример из конспекта — чтение целого числа из файла:

```
std::ifstream fin("input.txt");
int a;
fin >> a;
```

Разберём построчно:

1. `std::ifstream fin("input.txt");` — создаётся объект `fin` типа `std::ifstream`, который сразу же при создании открывает файл `input.txt` для чтения. Имя `fin` здесь не ключевое слово, а просто выбранное имя переменной (сокращение от “file

`in`”), но такое имя часто используют по соглашению именно для входного файлового потока.

2. `int a;` — объявляется обычная переменная `a`, в которую мы собираемся считать значение.
3. `fin >> a;` — оператор `>>` (“оператор извлечения из потока”) читает из потока `fin` очередное значение и помещает его в `a`. Поскольку `a` имеет тип `int`, поток попытается прочитать из файла текст, интерпретируемый как целое число, и преобразовать его в `int`.

Здесь стоит обратить внимание на аналогию с уже знакомым `std::cin >> a;` — работа с `std::ifstream` устроена так же, как со стандартным потоком ввода: тот же оператор `>>`, та же логика чтения по значению. Разница лишь в том, откуда именно берутся данные — из файла, а не с клавиатуры/стандартного ввода.